

MetaSignal

Upgraded PRD v2.0

Production-Simulated Metrics Intelligence & Experiment Decision Platform

Version 2.0 · April 2026 · Solo-built, Non-production, Production-simulated
Target: Senior/Staff DS · Product Scientist · Decision Systems — FAANG-level

Version: 2.0 — April 2026

Build Archetype: Solo-built, non-production, production-simulated flagship

Target Signal: Senior/Staff DS · Product Scientist · Decision Systems — FAANG-level

1. Executive Summary

What MetaSignal Is Now

MetaSignal is a solo-built, production-simulated metrics intelligence and experiment decision platform. It ingests synthetic event data through a realistic failure-injected pipeline, computes versioned business metrics with auditable denominator logic, detects anomalies with day-of-week seasonal correction and validated precision/recall, evaluates experiments using CUPED variance reduction and pre-defined guardrail gates, and records every ship/hold/investigate decision with mandatory human review and override justification. The system's operational history — pipeline failures, recovered backfills, false-positive anomaly feedback, guardrail breaches, delayed guardrail violations, human override decisions — is scripted to simulate 60 days of real platform operation. Every design decision is defensible at the mechanism level.

Why It Exists

Most analytics stacks fail in three specific ways simultaneously: denominator inconsistency (conversion rate means different things to different teams), experiment invalidity (results read without checking power, guardrails, or novelty effects), and decision unauditability (nobody knows why a metric moved or who decided to ship). MetaSignal is the infrastructure that prevents these failures. It is not built because these problems are rare — it is built because every product company running more than five experiments per quarter experiences all three, and companies like Statsig, Amplitude, and Eppo charge \$50k–\$500k/year to solve them.

What Hiring Signal It Creates

Alone, MetaSignal demonstrates: metric design rigor, experimentation judgment, causal reasoning awareness, data pipeline observability, decision quality thinking, and scale-aware data engineering. Combined with DevPulse (AI engineering, retrieval systems, LLM orchestration), the two projects cover the complete hiring signal map for FAANG-level DS, Product Scientist, and Decision Systems roles. A hiring manager who looks at both sees a profile that is unusually complete: someone who can build AI systems and reason about analytics infrastructure simultaneously — a combination that is rare and specifically targets L5/L6 DS roles.

Why Production-Simulation Matters

The gap between "production-shaped" (architecture looks right) and "production-simulated" (the system has encountered and handled real operational failure modes, and the evidence is in the logs) is the gap between a portfolio project and a credible hiring signal. A staff-level interviewer does not ask "how does your architecture look?" They ask: "What happens when T-1 data arrives late and you've already computed that day's metrics?" "How do you know your anomaly detector isn't just firing on Mondays?" "What does your decision log look like when a human overrides the recommendation?" MetaSignal answers these questions with real database entries, not correct schemas.

2. Current PRD Diagnosis

What Is Already Strong (Preserve Exactly)

- **Schema design:** Denominator_sql as required field, config_hash per pipeline run, parent_version_id for metric lineage, false_positive feedback on anomaly_alerts — genuine engineering decisions, not cosmetic.
- **Experimentation design (Section 9 of v1):** Correct CUPED derivation, comprehensive causal traps table (novelty effect, SUTVA, Simpson's, peeking, survivorship), deterministic decision engine logic.
- **Interview Q&A (15 questions):** Particularly strong: Q11 (CUPED walkthrough), Q12 (peeking problem), Q13 (Simpson's), Q15 (at Amazon's scale). Preserve verbatim.
- **Tech stack rationale:** DuckDB-over-Spark justification is architecturally correct. SQL-portable dbt logic argument is a genuine insight.
- **Scale story:** Three-tier reasoning (1–5M, 50M, 500M events/day) with SQL-portable transforms is interview-ready.
- **Red-team section:** Risks 1–4 identify real failure modes. Preserve and extend.
- **Day-by-day roadmap structure:** Pipeline-first sequencing is correct.

What Is Insufficient (Must Be Fixed)

Blank sections (critical gaps):

- Section 3 of v1: "Primary users:", "Secondary users:", "Jobs to be done:", "Core workflows:", "Pain points:" — all empty headers.
- Section 4 of v1: Non-LLM Baseline — four subheadings, no content.
- Section 5 of v1: Scope Definition — MVP/V1/Stretch/Excluded all empty.
- Section 10 of v1: Offline validation specifics — blank.
- Section 13 of v1: Security/Privacy — four subheadings, empty.
- Section 17 of v1: "Must exist / Cut first / Cut second" — blank.

Missing simulation design: The data generator is mentioned as "synthetic e-commerce events, ~2M rows" with no failure injection design. The 60-day operational history that makes this production-simulated is entirely absent.

Missing golden scenario suite: Named but not defined. What are the 12 scenarios? What does each prove?

Missing demo narrative: "Run script, show output" is not a demo. A believable operational sequence is needed.

CUPED A/A validation underspecified: Claiming false positive rate control requires demonstrating it with a measured result, not stating it.

Technical Errors in v1 (Must Be Corrected)

1. **Bonferroni misapplied to guardrails** — guardrails are mechanical gates, not hypothesis tests requiring multiple comparison correction. See Section 10 for the corrected approach.
2. **Guardrail evaluation ordering not explicit** — guardrails must be evaluated BEFORE the primary metric ship decision, not after.
3. **Right-censoring absent** — delayed metrics (returns, refunds, cancellations) must be flagged as immature at early readout. The most important demo scenario (Day 60 retroactive guardrail breach) depends on this.
4. **CUPED edge cases not handled** — weak covariate, missing X_pre, zero variance in X_pre, imbalanced arms.
5. **Anomaly backtest overclaims** — precision/recall must be labeled as "validated on synthetic injected events" with honest difficulty distribution.
6. **Demo metrics too simple** — denomination traps must be visible in actual metric definitions, not just claimed.

3. Product Positioning

One-Line Pitch

MetaSignal is a versioned metrics intelligence and experiment decision platform that makes the denominator explicit, the evaluation rigorous, and every product decision auditable — replacing the fragmented, unversioned, and unauditable state of most analytics stacks.

What MetaSignal Is Not

- Not a production system running on real company data
- Not a Tableau dashboard or Metabase alternative
- Not a notebook-based analysis
- Not a "what if we built Statsig" fantasy with shallow implementation
- Not a system with fake precision/recall numbers presented as real production metrics
- Not frontend-first

What MetaSignal Is

- A system with a correct statistical core: CUPED validated on synthetic A/A data with measured false positive rate, anomaly detection labeled and calibrated, guardrail logic correctly ordered
- A system with operational history showing failure modes encountered and handled: pipeline_run_log contains blocked runs, backfill recovery entries, retry logs
- A system with a decision audit trail containing realistic non-trivial entries: human overrides with reasons, retroactive guardrail violations, delayed guardrail flags
- A system where the denominator design story is visible in the actual metric definitions — three definitions of "conversion rate" with different denominators, all active, all flagged by the conflict detector nightly
- A system where scale reasoning is embedded in the architecture, not bolted on after the fact

Why It Is Not a Toy Project

Three things distinguish this from a toy:

(1) The statistical implementation has been validated. CUPED on 1000 synthetic A/A runs shows approximately uniform p-value distribution (KS test against Uniform(0,1) passes). Anomaly detector precision/recall measured on a labeled set with deliberate difficulty distribution — not all obvious failures.

(2) The operational history has non-trivial entries. Schema drift blocked a compute job (logged). Late-arriving data triggered a partition recompute (logged, late_arrival_flag=TRUE on recomputed rows). A human overrode a hold recommendation with a written business justification (logged, immutable). A delayed guardrail violated at Day 60 what looked safe at Day 14 (logged, retroactive entry in anomaly_alerts).

(3) The design decisions are explainable at the mechanism level. Not "I used CUPED" but: "CUPED works because pre-experiment variance is noise from the experiment's perspective — we subtract the predictable component using $\theta = \text{Cov}(Y_{\text{post}}, X_{\text{pre}}) / \text{Var}(X_{\text{pre}})$. When the covariate is weakly correlated ($|\theta| < 0.05$), CUPED adds more noise than it removes, so we fall back to unadjusted." Not "I check guardrails" but: "Guardrails are not hypothesis tests — they are pre-defined mechanical gates evaluated against a tolerance threshold before the primary metric decision, because harm is asymmetric."

4. User and Business Context

Primary Users

Data Scientists and Experimentation Analysts at product companies running more than 5 experiments per quarter. Daily pain: experiment results that may be invalid (insufficient power, peeking, guardrail violations unchecked), anomalies discovered by whoever happens to notice a dashboard number looks wrong, and metric definitions that mean different things across teams.

Growth and Product Engineers who need to ship experiments confidently without requesting a bespoke analysis from a DS. They need a structured, statistically correct recommendation with all guardrails checked — not a p-value and a judgment call.

Secondary Users

Product Managers who need a structured record of which experiments shipped, what the evidence said, and who made the final call — for retrospectives, for post-launch validation, and for explaining metric movements to leadership.

Data Engineering Teams responsible for pipeline reliability who need a structured system for logging quality failures, triggering recomputes, and tracking schema drift.

Jobs to Be Done

1. "I need to know if this experiment is ready to ship — give me a structured, statistically correct recommendation with all guardrails checked and documented."
2. "Our DAU dropped 8% yesterday — tell me whether it's a real metric movement, a data quality failure, or a seasonal artifact, and show me the top contributing segment."
3. "The definition of 'active user' changed last quarter — show me what our historical metrics look like under the old and new definitions so I can understand the denominator impact."
4. "We shipped a feature three months ago — tell me what the primary metric actually did post-launch versus what we predicted, and whether any guardrails violated after maturity."
5. "Audit Q3: show me every experiment decision made, who made it, what the system recommended, and whether any human overrides were later validated or proven wrong."

Recurring Workflows

- **Daily:** Pipeline runs quality checks, computes metrics, detects anomalies. Failures are logged and alerted.
- **On experiment readout:** DS submits experiment for evaluation. System runs CUPED, Welch t-test, guardrail gates (before primary decision), peeking check, Simpson's check. Produces structured recommendation.
- **On anomaly alert:** Analyst reviews, confirms or marks false positive. Feedback updates precision/recall tracking.
- **On metric definition change:** New version created, old deprecated, backfill job computes 90 days of history under new definition.
- **30/60 days post-ship:** PostLaunchValidationService checks delayed guardrails at maturity, compares primary metric prediction to realized outcome.

Pain Points

- Denominator of "conversion rate" means different things to growth, product, and finance. No system enforces the authoritative definition.
- A/B tests are evaluated by looking at the dashboard whenever someone feels like it — peeking inflates false positive rate with no record of how many times the result was checked.
- Anomalies are discovered in Slack when someone notices a number looks wrong, not by a systematic detection system.
- No record of why metrics moved, which version of a metric was in effect during which experiment, or whether decisions were made on sound statistical grounds.

- Post-launch validation almost never happens — shipped features' actual impact is rarely measured against pre-ship predictions.

Why a Company Would Care

Any company running more than 5 experiments per quarter with more than 3 analysts defining metrics is experiencing this problem. One bad experiment ship decision based on an unchecked guardrail violation costs more than building this system. Companies like Statsig, Amplitude, and Eppo charge \$50k–\$500k/year to solve this for enterprise customers.

What Bad Decisions This Prevents

- Shipping a feature with positive primary metric lift but a guardrail violation in latency or return rate (harm not caught)
- Evaluating an experiment after 3 days when it was designed for 14 (peeking-driven false positive)
- Using the wrong denominator for conversion rate in an experiment, making treatment look better than it is
- Reporting "we improved DAU by 4%" when the denominator definition changed between measurement and baseline
- Never knowing whether a shipped feature actually delivered on its predicted metric improvement

5. Production-Simulated System Definition

What Is Real

- **Statistical implementation:** CUPED formula is correct NumPy/Pandas code. Welch's t-test with correct degrees of freedom. Guardrail gates evaluated before primary metric. Right-censoring logic with maturity window. DOW-adjusted Z-score anomaly detection. Edge case handlers for CUPED (weak covariate, missing X_pre, zero variance, imbalanced arms).
- **Data model:** All schemas in Section 7 are real Postgres tables with Alembic migrations.
- **Pipeline logic:** Quality checks run before compute. Blocking failures halt compute, log error detail, trigger alerts. Retry with exponential backoff. All runs logged to pipeline_run_log.
- **Audit trail:** Every metric compute run has a config_hash. Every experiment evaluation records evaluator_version. Every decision requires reviewer name and — if overriding — a written override_reason. Decision log is append-only.
- **Validation:** CUPED validated on synthetic A/A data (1000 runs, p-value uniformity). Anomaly detector precision/recall measured on labeled synthetic set with deliberate difficulty distribution (documented in README as synthetic-labeled, not real production data).

What Is Simulated

- **Event data:** Configurable Python generator producing partitioned Parquet files mimicking an e-commerce product (page_view, pdp_view, add_to_cart, checkout, purchase, return_initiated, subscription events). Synthetic but volumetrically realistic.
- **Failure events:** Schema drift, null spikes, late-arriving data, double-counting — injected as scripted scenarios on specific dates, documented in simulation_events table.
- **Experiment assignment:** Deterministic hash-based splits in the synthetic generator, not real random user assignment.
- **Human review decisions:** Pre-populated in decision_log by the demo seed script. Reviewer names are fictional. Override reasons are written by the builder.
- **Anomaly ground truth:** The 20-event backtest set is synthetic with injected anomalies, labeled as such.
- **Post-launch outcomes:** Scripted to match predictions for some experiments (system was right) and diverge for others (delayed guardrail breach).

What Is Logged

Every pipeline run, quality check result, anomaly alert, experiment evaluation, decision, backfill job, and system health summary produces structured JSON log entries and database table rows. The system's operational state is fully reconstructable from the database.

What Is Validated

- CUPED false positive rate on 1000 synthetic A/A runs (uniform p-value distribution, KS test, result committed to README)
- CUPED variance reduction percentage (measured per experiment, stored in `experiment_evaluations.variance_reduction_pct`)
- CUPED edge cases: all four edge case types tested in golden scenario suite
- Anomaly detector: precision, recall, F1 per anomaly category on labeled synthetic set
- Decision engine: 12 golden scenario cases with known expected outputs, run in CI

What Is Explicitly Not Claimed

- This is not a production system
- Event data is entirely synthetic — no real users, sessions, or transactions
- Anomaly precision/recall is measured on synthetic injected events, not real historical data
- The system has not been tested under real concurrent load
- Human override decisions were written by the builder

6. Revised Architecture

```

DEMO RUNNER LAYER ■
seed_demo.py — generates 60-day operational history in sequence ■
show_demo_report.py — terminal evidence summary for narration ■

DATA GENERATION / SIMULATION LAYER ■
generate_events.py — partitioned Parquet e-commerce events ■
inject_failures.py — schema drift, null spikes, late-arriving, dedup ■
generate_experiments.py — hash-based assignment, outcome injection ■
generate_anomaly_labels.py — labeled ground truth for backtesting ■

INGESTION LAYER ■
ingest_job.py — validate Parquet schema vs stored snapshot, partition ■
Schema validator — compare against schema_snapshots in Postgres ■

STORAGE LAYER ■
Parquet (partitioned by event_date / country) — raw event data ■
PostgreSQL (Neon free tier) — registry, audit, experiments, decisions ■

METRIC REGISTRY LAYER ■
MetricRegistryService — versioned definitions, config_hash, lineage ■
MetricConflictDetectorService — nightly denominator divergence check ■

```

```
■ CI / TESTING LAYER (GitHub Actions) ■
■ golden_scenario_suite.py – 12 deterministic evaluation engine tests ■
■ cuped_aa_validation.py – 1000 A/A runs, p-value uniformity KS test ■
■ anomaly_backtest.py – precision/recall on labeled set ■
```


Interview signal: run_id linkage traces any result to the exact pipeline run. quality_passed filters out metrics computed on bad data. late_arrival_flag marks recomputed rows.

```
SQL
id UUID PRIMARY KEY
metric_id UUID REFERENCES metric_definitions(id)
metric_version INTEGER NOT NULL
computation_date DATE NOT NULL
grain_period TEXT NOT NULL -- "2024-01-15"
numerator_value FLOAT
denominator_value FLOAT
metric_value FLOAT -- numerator / denominator
segment_key TEXT NOT NULL DEFAULT 'ALL' -- "country=IN"|"platform=ios"|"ALL"
run_id UUID REFERENCES pipeline_run_log(id)
quality_passed BOOLEAN NOT NULL
late_arrival_flag BOOLEAN DEFAULT FALSE -- TRUE if recomputed after late-arriving data
created_at TIMESTAMPTZ DEFAULT now()
```

pipeline_run_log

Why it exists: Immutable audit trail of every pipeline execution. This table is evidence of operational history — an interviewer can query it and see real failures, recoveries, and recomputes.

Interview signal: status="blocked_by_quality" entries prove the data quality gate actually fires. retry_count shows exponential backoff in action.

```
SQL
id UUID PRIMARY KEY
job_type TEXT NOT NULL -- "ingest"|"quality_check"|"metric_compute"|"anomaly_detection"|"backfill"
metric_id UUID -- NULL for ingest/quality jobs
run_date DATE NOT NULL
started_at TIMESTAMPTZ NOT NULL
completed_at TIMESTAMPTZ
status TEXT NOT NULL -- "success"|"failed"|"partial"|"blocked_by_quality"
rows_processed INTEGER
rows_expected INTEGER
quality_checks_passed INTEGER
quality_checks_failed INTEGER
error_message TEXT
config_hash TEXT -- metric definition hash at run time
retry_count INTEGER DEFAULT 0
triggered_by TEXT DEFAULT 'scheduler' -- "scheduler"|"backfill_job"|"manual"
created_at TIMESTAMPTZ DEFAULT now()
```

data_quality_checks

Why it exists: Stores individual check results per run. Enables querying which check type fails most often and on which entities.

Interview signal: check_before_compute discipline. Serving a metric computed on bad data is worse than serving no metric.

```
SQL
id UUID PRIMARY KEY
run_id UUID REFERENCES pipeline_run_log(id)
check_name TEXT NOT NULL -- "row_count_stability"|"null_rate"|"schema_match"|"freshness"|"dedup"
entity TEXT NOT NULL -- "events"|"user_properties"
check_date DATE NOT NULL
expected_value FLOAT
actual_value FLOAT
threshold FLOAT
```

```
passed BOOLEAN NOT NULL
severity TEXT NOT NULL -- "warning"|"blocking"
check_details JSONB -- {"column": "entity_id", "null_pct": 0.34, "threshold": 0.05}
created_at TIMESTAMPTZ DEFAULT now()
```

anomaly_alerts

Why it exists: Tracks every fired alert, acknowledgment, and false-positive feedback. The feedback loop turns this into a labeled dataset for continuous detector improvement.

Interview signal: false_positive flag + feedback_notes makes decision quality measurable. injected_anomaly flag maintains honest labeling of backtest events.

```
SQL
id UUID PRIMARY KEY
metric_id UUID REFERENCES metric_definitions(id)
detection_date DATE NOT NULL
metric_value FLOAT
expected_value FLOAT
z_score FLOAT
ewma_deviation FLOAT
dow_adjusted BOOLEAN DEFAULT TRUE
severity TEXT NOT NULL -- "low"|"medium"|"high"|"critical"
detection_method TEXT NOT NULL -- "z_score_dow"|"ewma"|"both"
attribution JSONB -- {"top_segment": "country=IN", "contribution": 0.72}
acknowledged BOOLEAN DEFAULT FALSE
acknowledged_by TEXT
acknowledged_at TIMESTAMPTZ
false_positive BOOLEAN
feedback_notes TEXT
injected_anomaly BOOLEAN DEFAULT FALSE -- TRUE if injected for backtesting
created_at TIMESTAMPTZ DEFAULT now()
```

experiments

Why it exists: Registry of all experiment configurations. MDE calculator at registration prevents underpowered experiments from starting.

Interview signal: guardrail_metric_configs as JSONB stores per-guardrail tolerance AND maturity window, enabling right-censoring logic.

```
SQL
id UUID PRIMARY KEY
name TEXT NOT NULL
hypothesis TEXT NOT NULL
start_date DATE NOT NULL
end_date DATE
status TEXT NOT NULL -- "draft"|"running"|"paused"|"completed"|"stopped_guardrail"
primary_metric_id UUID REFERENCES metric_definitions(id)
guardrail_metric_configs JSONB -- [{"metric_id": uuid, "tolerance_pct": -0.015, "maturity_days": 60,
"direction": "lower_is_better"}]
expected_mde FLOAT
expected_duration_days INTEGER NOT NULL
alpha FLOAT DEFAULT 0.05
power FLOAT DEFAULT 0.80
use_cuped BOOLEAN DEFAULT TRUE
assignment_method TEXT DEFAULT 'deterministic_hash'
treatment_split_pct FLOAT DEFAULT 0.50
notes TEXT
created_by TEXT NOT NULL
created_at TIMESTAMPTZ DEFAULT now()
```

experiment_evaluations

Why it exists: Stores each evaluation run. Multiple evaluations per experiment enable peeking detection.

Interview signal: cuped_theta, variance_reduction_pct, and cuped_edge_case fields are proof the CUPED implementation is real and handles edge cases. guardrails_cleared is distinct from is_significant — they have different preconditions.

```
SQL
id UUID PRIMARY KEY
experiment_id UUID REFERENCES experiments(id)
evaluated_at TIMESTAMPTZ NOT NULL
evaluation_day INTEGER
sample_size_treatment INTEGER NOT NULL
sample_size_control INTEGER NOT NULL
primary_metric_lift FLOAT
primary_metric_ci_lower FLOAT
primary_metric_ci_upper FLOAT
p_value FLOAT
is_significant BOOLEAN
cuped_applied BOOLEAN
cuped_theta FLOAT -- stored for auditing
variance_reduction_pct FLOAT -- MEASURED, not assumed
cuped_edge_case TEXT -- NULL|"weak_covariate"|"missing_xpre"|"no_variance"|"imbalanced_arms"
cuped_fallback TEXT -- NULL|"unadjusted"
guardrail_results JSONB -- [{"metric_id": uuid, "name": str, "lift": float, "ci_lower": float, "ci_upper": float, "status": "clear"|"violated"|"immature", "maturity_days_remaining": int}]
guardrails_cleared BOOLEAN -- ALL guardrails passed (not violated, not immature)
peeking_warning BOOLEAN DEFAULT FALSE
simpsons_flag BOOLEAN DEFAULT FALSE
segment_analysis JSONB
novelty_flag BOOLEAN DEFAULT FALSE
decision_recommendation TEXT NOT NULL -- "ship"|"hold"|"investigate"|"wait_guardrail_maturity"
recommendation_reason TEXT NOT NULL
evaluator_version TEXT NOT NULL
is_aa_test BOOLEAN DEFAULT FALSE
created_at TIMESTAMPTZ DEFAULT now()
```

decision_log

Why it exists: Immutable human-in-the-loop record. The system cannot ship without a decision entry. Override reason required when human overrides system.

Interview signal: The override_reason enforcement is the design decision that makes decision quality measurable over time. post_launch_validation_due triggers the delayed validation flow.

```
SQL
id UUID PRIMARY KEY
evaluation_id UUID REFERENCES experiment_evaluations(id)
reviewer TEXT NOT NULL
system_recommendation TEXT NOT NULL
human_decision TEXT NOT NULL -- "ship"|"hold"|"investigate"|"override_ship"|"override_hold"
override_reason TEXT -- REQUIRED when human_decision != system_recommendation (enforced in API)
risk_acknowledgment TEXT
post_launch_validation_due DATE -- 30 days after ship or override_ship decision
decided_at TIMESTAMPTZ NOT NULL
```

backfill_jobs

Why it exists: Tracks partition-level recompute jobs. Enables answering: was this metric result computed on original data or recomputed after a late-arrival?

Interview signal: Incremental recompute — you don't recompute all 90 days when T-1 data arrives late.

```
SQL
id UUID PRIMARY KEY
metric_id UUID REFERENCES metric_definitions(id)
trigger_reason TEXT NOT NULL --
"late_arriving_data"|"quality_failure_recovery"|"definition_version_change"
affected_dates DATE[] NOT NULL
triggered_by_run_id UUID REFERENCES pipeline_run_log(id)
status TEXT NOT NULL -- "queued"|"running"|"complete"|"failed"
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
rows_recomputed INTEGER
created_at TIMESTAMPTZ DEFAULT now()
```

metric_conflicts

Why it exists: Nightly detection of denominator/filter divergence across the metric registry. The conflict detector must actually fire on the demo data.

Interview signal: Three conflicting conversion rate definitions are deliberately designed into the registry. If the detector never fires, the denominator story is abstract, not demonstrated.

```
SQL
id UUID PRIMARY KEY
metric_a_id UUID REFERENCES metric_definitions(id)
metric_b_id UUID REFERENCES metric_definitions(id)
conflict_type TEXT NOT NULL -- "denominator_mismatch"|"filter_divergence"|"entity_grain_mismatch"
value_divergence_pct FLOAT
sample_period TEXT
detected_at TIMESTAMPTZ NOT NULL
resolved BOOLEAN DEFAULT FALSE
resolution_notes TEXT
```

simulation_events

Why it exists: Documents every injected failure scenario. Reproducibility proof — the demo state is exactly reproducible from a fresh database using this table.

Interview signal: Complete transparency about what was simulated and when. No hidden assumptions.

```
SQL
id UUID PRIMARY KEY
event_type TEXT NOT NULL -- "schema_drift"|"null_spike"|"late_arriving"|"anomaly_injection"|"experiment_start"|"guardrail_breach"|"human_override"
event_date DATE NOT NULL
description TEXT NOT NULL
affected_entity TEXT
injected_value JSONB -- what was injected and expected system response
created_at TIMESTAMPTZ DEFAULT now()
```

system_health_reports

Why it exists: Daily summary of operational health. Demonstrates that decision quality is itself measured, not just asserted.

Interview signal: decision_override_rate as a system health metric is the insight that the system's calibration is measurable over time.

```
SQL
id UUID PRIMARY KEY
report_date DATE NOT NULL
pipeline_success_rate FLOAT
metrics_fresh_count INTEGER
metrics_stale_count INTEGER
```

```

anomaly_alerts_fired INTEGER
anomaly_alerts_acknowledged INTEGER
anomaly_false_positive_rate FLOAT -- rolling 30-day rate
experiment_evaluations_completed INTEGER
decisions_made INTEGER
decisions_overridden INTEGER
decision_override_rate FLOAT
post_launch_validations_due INTEGER
post_launch_validations_completed INTEGER
post_launch_validation_failures INTEGER
notes TEXT
created_at TIMESTAMPTZ DEFAULT now()

```

8. Synthetic Data and Failure Injection Design

Event Generator Design

The core generator produces partitioned Parquet files simulating a fictional B2C SaaS product with a checkout funnel. Behavioral realism:

- **Daily volume:** 200K–2M events/day varying by DOW (Mon–Thu higher, Fri–Sun lower, with stochastic noise)
- **Event types:** `page_view`, `pdp_view`, `add_to_cart`, `checkout_start`, `purchase`, `return_initiated`, `subscription_start`, `subscription_cancel`
- **Entity fields:** `user_id` (persistent UUID), `session_id` (per-visit UUID)
- **Dimensions:** `country` (US 40%, IN 25%, UK 15%, DE 10%, other 10%), `platform` (web 55%, ios 30%, android 15%)
- **Experiment fields:** `experiment_id`, `variant` (control/treatment/NULL)
- **Partition layout:** `/events/year=YYYY/month=MM/day=DD/country=CC/events_YYYYMMDD_CC_NNN.parquet`

Scripted Failure Scenarios (13 Injection Events)

Scenario 1: Clean Baseline (Days 1–10)

Normal data generation. All quality checks pass. Metrics compute daily. No anomalies. Establishes the DOW baseline distribution required for anomaly detection. System health: `pipeline_success_rate = 1.0`.

Scenario 2: Schema Drift Blocking Compute (Day 11)

Generator drops the `country` column from Day 11 Parquet (simulating upstream schema change). Schema validator detects mismatch against stored snapshot. DataQualityService records blocking `schema_match` failure. Compute job halted. `pipeline_run_log: status = "blocked_by_quality", error_message = "Schema mismatch: column 'country' missing"`. Metric freshness alert fires. `simulation_events: event_type = "schema_drift"`.

Scenario 3: Recovery and Backfill (Day 12)

Country column restored. Day 12 pipeline succeeds. BackfillService triggered for Day 11 partitions. Two `pipeline_run_log` entries for Day 11: original failed + backfill success. Day 11 `metric_results` rows: `late_arrival_flag = TRUE`. Day 12 rows: `late_arrival_flag = FALSE`.

Scenario 4: Null Spike Blocking Compute (Day 15)

`entity_id` null rate spikes to 38% for US platform (simulating logging bug). DataQualityService detects 19x baseline. Records blocking failure: `check_details = {"column": "entity_id", "null_pct": 0.38, "threshold": 0.05, "baseline_7d": 0.02}`. Compute halted. Resolved Day 16 with backfill.

Scenario 5: Late-Arriving Data Recompute (Day 20)

Day 19 partition receives 15% additional rows on Day 20 (ETL delay). Row count delta for Day 19 triggers BackfillService. Two metric_results rows for Day 19: original (rows_processed = 1.2M) and recomputed (rows_processed = 1.38M, late_arrival_flag = TRUE).

Scenario 6: Double-Counting Blocking Compute (Day 25)

Row count for Day 25 is 3.1x the 7-day average (duplicate event ingestion). DataQualityService dedup check fires as blocking. investigation_note logged. Resolved Day 26 with deduplicated reingestion.

Scenario 7: Real Anomaly — DAU Drop (Day 28)

DAU drops 14% (scripted iOS platform outage). DOW-adjusted Z-score = 3.8. Alert: severity = "high", attribution = {"top_segment": "platform=ios", "contribution": 0.81, "method": "counterfactual"}. Analyst priya_sharma acknowledges: false_positive = FALSE. injected_anomaly = TRUE (for backtest labeling).

Scenario 8: False Positive — DOW Suppressed (Day 35)

Long weekend effect. Raw Z-score = 2.7 (would fire). DOW-adjusted Z-score = 1.1 (below threshold — suppressed correctly). Anomaly does NOT fire. Demo shows raw vs. adjusted comparison. This is the key proof that DOW adjustment works. In the backtest labeled set, this event type is marked false_positive = TRUE.

Scenario 9: Metric Definition Change (Day 40)

active_users_v1 deprecated: counted any session event. active_users_v2 created: counts users with ≥1 pdp_view (meaningful engagement). BackfillService recomputes 90 days of v2. ConflictDetector nightly run flags the pair: conflict_type = "filter_divergence", value_divergence_pct = 0.23 (v2 is 23% lower — many passive visitors excluded). Both versions visible in metric registry with SQL diff.

Scenario 10: Experiment EXP-001 Start — Checkout Redesign (Day 30)

Assignment: deterministic hash(user_id + "EXP-001") % 100 < 50. Synthetic outcome injection: treatment group has purchase_rate +3.5%, page_load_time improvement -12ms, return_initiated_rate +1.8% (delayed, observable at Day 60 only). MDE check at registration: "expected effect of +3.5% is above the +1.4% minimum detectable at 80% power over 14 days."

Scenario 11: Delayed Guardrail Simulation — EXP-001 (Days 44 + 60)

Day 44 evaluation: primary metric p=0.023, CI=[+1.2%, +5.8%], CUPED variance reduction 31.2%. page_load_time guardrail: mature, cleared. return_rate guardrail: status = "immature" (day 14 of 60-day window). Decision: "wait_guardrail_maturity".

Day 60 post-launch check: return_rate matured CI = [+0.4%, +2.1%] against tolerance +1.5%. **Retroactive guardrail violation.** anomaly_alerts entry inserted: detection_method = "post_launch_validation". system_health_reports: post_launch_validation_failures = 1.

Scenario 12: Human Override (Day 44 — EXP-002)

EXP-002 evaluation: p=0.11, not significant. System: "hold". Reviewer james_okoye: human_decision = "override_ship", override_reason = "Week 4 marketing campaign created confound; segmented analysis shows +5.1% lift for returning users (p=0.009). PM accepted risk. Approved by product leadership." Logged, immutable.

Scenario 13: Clean Ship (Day 50 — EXP-003)

p=0.008, CI=[+2.1%, +7.3%], both guardrails mature and cleared, CUPED applied, no flags. decision_recommendation = "ship". Reviewer sara_lindqvist: human_decision = "ship". post_launch_validation_due = Day 80.

9. Metric Design

Six Core Metrics in the Registry

Metric 1: daily_active_users_v1 (deprecated, Day 40)

- Numerator SQL: `COUNT(DISTINCT entity_id) WHERE event_type IN ('page_view', 'pdp_view', 'add_to_cart', 'checkout_start', 'purchase')`
- Denominator: 1 (absolute count)
- Direction: higher_is_better
- Deprecation: includes passive visitors with no meaningful engagement

Metric 2: daily_active_users_v2 (active)

- Filter SQL: `WHERE event_type IN ('pdp_view', 'add_to_cart', 'purchase')` — meaningful engagement only
- Denominator: 1 (absolute count)
- **Conflict pair with v1:** filter_divergence, value_divergence_pct ≈ 0.23 — the conflict detector fires on this pair and inserts a metric_conflicts row nightly

Metric 3: checkout_conversion_rate_all_visitors (active, exploratory)

- Numerator SQL: `COUNT(DISTINCT CASE WHEN event_type='purchase' THEN entity_id END)`
- Denominator SQL: `COUNT(DISTINCT entity_id)` — all platform visitors
- Denominator type: `all_visitors`
- Denominator trap: inflates denominator with users who never entered the funnel

Metric 4: checkout_conversion_rate_pdp_visitors (active, authoritative)

- Numerator SQL: same as above
- Denominator SQL: `COUNT(DISTINCT CASE WHEN event_type='pdp_view' THEN entity_id END)` — users who reached product detail page
- Denominator type: `pdp_visitors`
- **Conflict pair with Metric 3:** denominator_mismatch, value_divergence_pct ≈ 0.18 (pdp_visitors denominator smaller → ratio higher)

Metric 5: checkout_conversion_rate_add_to_cart (active, exploratory)

- Denominator SQL: `COUNT(DISTINCT CASE WHEN event_type='add_to_cart' THEN entity_id END)`
- Denominator type: `add_to_cart_users`
- **All three conversion metrics conflict with each other.** ConflictDetector flags all three pairs nightly. This is the denominatortrap story demonstrated, not just claimed.

Metric 6: return_rate (guardrail, delayed)

- Numerator SQL: `COUNT(DISTINCT CASE WHEN event_type='return_initiated' THEN entity_id END)`
- Denominator SQL: `COUNT(DISTINCT CASE WHEN event_type='purchase' THEN entity_id END)`
- Direction: lower_is_better
- is_guardrail: TRUE
- guardrail_tolerance_pct: 0.015 — max acceptable increase of 1.5 percentage points
- **guardrail_window_days: 60** — the right-censoring guardrail. Early readout looks safe; mature readout breaches tolerance. This is the central delayed guardrail demo scenario.

Metric 7: page_load_time_p95 (guardrail, fast-maturing)

- Computed: p95 of (event_timestamp - page_request_timestamp) in milliseconds
- Direction: lower_is_better
- is_guardrail: TRUE
- guardrail_tolerance_pct: 0.05 (max 5% degradation)
- guardrail_window_days: 7 — observable in first week

Denominator Design Principle

Every ratio metric stores `denominator_sql` and `denominator_type` separately. The denominator must answer: "what is the universe of entities this metric is computed over?" The API rejects any ratio metric created without `denominator_sql`. The conflict detector compares active metric pairs and flags divergence > 5% as a conflict.

10. Experimentation and Decision Design

Experiment Assignment

Deterministic hash-based: `hash(user_id + experiment_id) % 100 < split_pct * 100`. Reproducible without storing per-user assignment. Assignment is recorded in the event data generator as `experiment_id` and `variant` fields on relevant events. This design means assignment can be audited retroactively from the event data.

Multiple Comparison Correction Rules (Technical Correction from v1)

Primary metric: Standard alpha (default 0.05). If testing multiple primary metrics simultaneously or multiple variants, apply Bonferroni correction ($\alpha_{\text{adjusted}} = \alpha / k$) to those primary comparisons only.

Guardrail metrics: NOT hypothesis tests. Do NOT apply Bonferroni or any multiple comparison correction to guardrails. Guardrail metrics are pre-defined mechanical safety gates evaluated against a direction-and-tolerance threshold. The question is not "is this metric significantly different?" — it is "has this metric moved past the pre-defined acceptable harm threshold?" This is a deterministic gate, not a statistical inference.

Why this matters: Applying Bonferroni to guardrails is statistically incorrect — it would reduce the sensitivity of the safety mechanism by raising the threshold for declaring a violation. A guardrail that fires at +1.5% return rate increase should fire regardless of how many other guardrails are being evaluated simultaneously.

Guardrail Gate Logic

PYTHON

```
def evaluate_guardrail(guardrail_config, eval_result_for_metric, evaluation_day):
    """
    Guardrail evaluation: mechanical gate, not a hypothesis test.
    Returns: ("clear" | "violated" | "immature", reason_str)
    """
    # Step 1: Check maturity
    if evaluation_day < guardrail_config["maturity_days"]:
        days_remaining = guardrail_config["maturity_days"] - evaluation_day
        return "immature", f"Guardrail '{guardrail_config['name']}' not mature: {days_remaining} days remaining"

    tolerance = guardrail_config["tolerance_pct"]
    ci_lower = eval_result_for_metric["ci_lower"]
    ci_upper = eval_result_for_metric["ci_upper"]

    # Step 2: Direction-aware gate
    if guardrail_config["direction"] == "lower_is_better":
        # Violated if the LOWER bound of lift CI exceeds tolerance
        # (i.e., even the optimistic estimate shows harm beyond threshold)
        if ci_lower > tolerance:
            return "violated", f"CI lower bound {ci_lower:.3f} exceeds harm tolerance {tolerance:.3f}"
    elif guardrail_config["direction"] == "higher_is_better":
        # Violated if the UPPER bound of lift CI is below negative tolerance
```

```

        if ci_upper < -tolerance:
            return "violated", f"Negative movement CI upper bound {ci_upper:.3f} below tolerance {-tolerance:.3f}"

    return "clear", "Guardrail passed"

```

Decision Logic (Guardrails Evaluated BEFORE Primary Metric)

```

PYTHON
def compute_recommendation(eval_result):
    """
    Guardrail gates are evaluated BEFORE the primary metric decision.
    Any immature or violated guardrail takes precedence.
    """
    # Step 1: Immature guardrails — cannot conclude until all mature
    immature = [g for g in eval_result.guardrail_results if g["status"] == "immature"]
    if immature:
        names = [g["name"] for g in immature]
        return "wait_guardrail_maturity", f"Guardrails not yet mature: {names}"

    # Step 2: Violated guardrails — block ship regardless of primary metric
    violated = [g for g in eval_result.guardrail_results if g["status"] == "violated"]
    if violated:
        names = [g["name"] for g in violated]
        return "investigate", f"Guardrail violated: {names}. Primary metric result does not override a guardrail breach."

    # Step 3: Primary metric evaluation (only reached if all guardrails clear)
    if eval_result.p_value > eval_result.alpha:
        return "hold", "Primary metric not yet statistically significant"
    if eval_result.primary_metric_ci_lower > 0:
        return "ship", "Positive lift with CI above zero; all guardrails cleared and mature"
    if eval_result.primary_metric_ci_upper < 0:
        return "hold", "Negative primary metric lift with CI below zero"
    return "investigate", "CI spans zero; lift not practically significant"

```

CUPED Implementation with Edge Cases

```

PYTHON
def apply_cuped(y_post, x_pre):
    """
    CUPED with four edge case handlers.
    Returns: (y_adjusted, theta, variance_reduction_pct, edge_case_flag, fallback_method)
    """
    has_xpre = ~np.isnan(x_pre)
    n_total = len(y_post)
    n_with_xpre = has_xpre.sum()

    # Edge case 1: Too much missing X_pre data
    if n_with_xpre < 0.5 * n_total:
        return y_post, None, 0.0, "missing_xpre", "unadjusted"

    y_fit = y_post[has_xpre]
    x_fit = x_pre[has_xpre]

    # Edge case 2: No variance in X_pre
    if np.var(x_fit) < 1e-10:
        return y_post, 0.0, 0.0, "no_variance_xpre", "unadjusted"

    theta = np.cov(y_fit, x_fit)[0, 1] / np.var(x_fit)

```

```

# Edge case 3: Weak covariate — CUPED adds noise, not signal
if abs(theta) < 0.05:
    return y_post, theta, 0.0, "weak_covariate", "unadjusted"

# Edge case 4: Imbalanced arms — CUPED still valid but note
edge_case = None
n_treatment = (has_xpre & (x_pre >= 0)).sum() # proxy; replace with actual arm labels
imbalance_ratio = n_treatment / n_total
if imbalance_ratio < 0.35 or imbalance_ratio > 0.65:
    edge_case = "imbalanced_arms" # proceed with CUPED but flag

# Apply adjustment (users with x_pre only)
x_mean = np.mean(x_fit)
y_cuped = y_fit - theta * (x_fit - x_mean)
var_reduction = max(0.0, 1.0 - (np.var(y_cuped) / np.var(y_fit)))

return y_cuped, theta, var_reduction, edge_case, "cuped_applied"

```

A/A Validation Protocol

The `cuped_aa_validation.py` script runs 1000 synthetic A/A experiments:

1. Generate synthetic users with pre-experiment covariate (X_{pre}) and post-experiment metric (Y_{post}) drawn from the same distribution for both "treatment" and "control"
2. Apply the same deterministic hash assignment
3. Apply CUPED and run Welch's t-test
4. Record p-value

Expected result: p-values approximately Uniform(0,1) under the null.

Test: KS test against Uniform(0,1) at $\alpha=0.05$.

Target: False positive rate ($p < 0.05$) $\approx$ 5% (expect ~47–53 out of 1000).

If false positive rate > 7%: CUPED implementation has a bug — check theta estimation and variance calculation.

Result committed to README as: A/A false positive rate, KS test p-value, p-value distribution histogram.

Right-Censoring and Delayed Guardrails

Some guardrail metrics are "delayed" — the true effect is not observable at the primary readout date. Return rate is the canonical case: a purchase experiment may look safe at Day 14, but returns take 30–60 days to fully materialize.

Implementation:

- `guardrail_window_days` on `metric_definitions` specifies maturity
- At evaluation, if `evaluation_day < guardrail_window_days` → guardrail status = "immature" → decision = "wait_guardrail_maturity"
- The system does not ship an experiment with immature delayed guardrails, even if primary metric is significant
- `PostLaunchValidationService` checks matured guardrails at their maturity date
- If mature guardrail is violated post-maturity → retroactive `anomaly_alerts` entry + `system_health_reports` flag

Demo simulation (EXP-001):

- Day 44: `return_rate` CI = [-0.8%, +2.4%] — spans zero, but `status` = "immature" (day 14 of 60-day window). Decision: "wait_guardrail_maturity".
- Day 60: `return_rate` matured CI = [+0.4%, +2.1%] against tolerance +1.5%. Retroactive guardrail violation. Logged.

Peeking Warning

Every evaluation run is stored in `experiment_evaluations`. If significance conclusion at `evaluation_day N` differs from any prior evaluation for the same experiment, `peeking_warning = TRUE`. Analyst UI shows evaluation history chart — p-value over days.

Novelty Effect Detection

If week-1 per-user metric lift > 2× week-2 per-user metric lift, `novelty_flag = TRUE`. Recommendation adds: "Novelty effect suspected — consider extending experiment to 4 weeks."

Post-Launch Validation

Scheduled by APScheduler at `post_launch_validation_due` date from `decision_log`. Computes: actual primary metric change post-ship vs pre-ship prediction CI. For delayed guardrails: re-evaluates gate with mature data. Calibration score tracked: how often positive predictions correlate with positive realizations. If consistently over-predicting, evaluation engine may be miscalibrated (novelty or SUTVA).

11. Golden Scenario Suite

12 deterministic test cases run in CI on every change to the evaluation engine. Each takes fixed synthetic input and asserts exact expected output.

Scenario 1: Clean Ship

- Input: +4.2% lift, $p=0.003$, $CI=[+1.8\%,+6.6\%]$, both guardrails mature+cleared, CUPED 28% reduction, no flags
- Expected: `decision_recommendation = "ship"`, `guardrails_cleared = TRUE`, `cuped_applied = TRUE`
- Proves: Happy path end-to-end

Scenario 2: Primary Lift + Guardrail Breach

- Input: +3.1% purchase rate, $p=0.04$, `page_load_time CI_lower = +6.2%` degradation (above 5% tolerance), guardrail mature
- Expected: `decision_recommendation = "investigate"`, reason includes guardrail name, `is_significant = TRUE` but blocked
- Proves: Guardrail gate fires BEFORE primary decision; positive primary CANNOT override guardrail

Scenario 3: Not Significant / Hold

- Input: +1.2% lift, $p=0.24$, $CI=[-0.9\%,+3.3\%]$, day 7 of 14-day planned experiment
- Expected: `decision_recommendation = "hold"`, `is_significant = FALSE`
- Proves: System waits for significance

Scenario 4: A/A False Positive Rate

- Input: 1000 synthetic A/A runs (identical distributions)
- Expected: false positive rate $\approx 5\%$ (47–53 out of 1000), KS test against $Uniform(0,1)$ passes
- Proves: CUPED implementation does not inflate false positive rate

Scenario 5: CUPED Strong Covariate

- Input: $\rho(Y_{\text{post}}, X_{\text{pre}}) = 0.72$, $\theta = 0.68$
- Expected: `cuped_applied = TRUE`, `variance_reduction_pct  $\approx 0.31$` , `cuped_edge_case = NULL`
- Proves: CUPED reduces variance when covariate is strong

Scenario 6: CUPED Weak Covariate

- Input: $\rho = 0.03$, $\theta = 0.02$ (< 0.05 threshold)

- Expected: `cuped_applied = FALSE`, `cuped_edge_case = "weak_covariate"`, `cuped_fallback = "unadjusted"`, `variance_reduction_pct = 0.0`
- Proves: System falls back gracefully when CUPED would add noise

Scenario 7: CUPED Missing X_pre

- Input: 60% of users have no pre-experiment data (new users)
- Expected: `cuped_applied = FALSE`, `cuped_edge_case = "missing_xpre"`, `cuped_fallback = "unadjusted"`
- Proves: Partial pre-experiment data handled without dropping users

Scenario 8: Delayed Guardrail — Immature at Readout

- Input: +3.8% lift, $p=0.01$, return_rate CI at day 14 = [-0.5%,+1.8%], maturity_days=60, evaluation_day=14
- Expected: `decision_recommendation = "wait_guardrail_maturity"`, `guardrail_status = "immature"`
- Proves: Right-censoring logic prevents premature ship

Scenario 9: Delayed Guardrail — Breach at Maturity

- Input: Same experiment, evaluation_day=60, return_rate CI=[+0.4%,+2.1%], tolerance=1.5%
- Expected: `decision_recommendation = "investigate"`, `guardrail_status = "violated"`
- Proves: Retroactive guardrail violation detected at maturity

Scenario 10: Novelty Effect Flag

- Input: Week-1 lift = +8.4%, week-2 lift = +1.2% (ratio = 7x)
- Expected: `novelty_flag = TRUE`, recommendation_reason includes novelty warning
- Proves: Novelty detection works

Scenario 11: Peeking Warning

- Input: Evaluations at days 3, 7, 10. Day 3: $p=0.048$. Day 7: $p=0.12$. Day 10: $p=0.034$
- Expected: `peeking_warning = TRUE` at day 10 (conclusion changed across evaluations)
- Proves: Peeking detection flags instability

Scenario 12: Human Override Enforcement

- Input: System recommendation = "hold", human_decision = "override_ship", override_reason = NULL
- Expected: API returns 422 Unprocessable Entity; override requires override_reason
- Input (valid): override_reason = "PM accepted risk per leadership review"
- Expected: decision_log entry written, `post_launch_validation_due` set
- Proves: Override reason is enforced in the API, not just documented

12. Anomaly and Monitoring Design

Day-of-Week Adjustment

```
PYTHON
def dow_adjusted_zscore(metric_history: dict, target_date: date) -> tuple[float | None, str]:
    """
    Compare target day to same-DOW historical distribution.
    Requires ≥4 weeks of same-DOW history.
    """
    dow = target_date.weekday()
    same_dow_values = [
        v for d, v in metric_history.items()
        if d.weekday() == dow
        and 7 <= (target_date - d).days <= 56 # 1-8 weeks back
    ]

    if len(same_dow_values) < 4:
```

```
        return None, "insufficient_dow_history"

    mu = np.mean(same_dow_values)
    sigma = np.std(same_dow_values)

    if sigma < 1e-10:
        return None, "zero_variance_history"

    z = (metric_history[target_date] - mu) / sigma
    return z, "dow_adjusted"
```

Alert Thresholds

Z-score	Severity
\	z\
\	z\
\	z\
\	z\

Medium alerts fire only if BOTH Z-score and EWMA flag the same metric on the same day (reduces false positives). Critical alerts fire on Z-score alone.

Anomaly Backtest Harness

The labeled set of 20 events has deliberate difficulty distribution:

Category	Count	Description
Obvious pipeline failures	5	50%+ row count drop, instantly detectable
Genuine metric movements	6	10–15% change, realistic signal
Ambiguous movements	4	5–8% change, debatable whether real or noise
Known DOW false positives	3	Weekend/holiday effects DOW adjustment should suppress
Delayed detection cases	2	Only detectable 2 days after the event

Precision/recall computed per category and reported honestly as:

"Validated on 20 synthetic injected anomaly events with deliberate difficulty distribution. This is not production precision/recall. These numbers represent the detector's behavior on synthetic data designed to test its calibration across anomaly types."

Honest Precision/Recall Claims

Can claim: "On the labeled synthetic set, the DOW-adjusted detector achieved precision X and recall Y. Raw Z-score without DOW adjustment had Y% higher false positive rate on the known seasonal events."

Cannot claim: "The system achieves X% precision in production." No production data exists.

Alert Lifecycle

1. Alert fires → `acknowledged = FALSE`
2. Analyst reviews → `acknowledged = TRUE`, `acknowledged_by`, `acknowledged_at` Set
3. Analyst marks → `false_positive = TRUE/FALSE`, `feedback_notes` written
4. Rolling 30-day false positive rate tracked in `system_health_reports`
5. If false positive rate > 0.40: `system_health_report` flags for threshold recalibration

13. Operational Flows

Flow 1: Scheduled Daily Metric Compute

```
07:00 UTC (APScheduler)
→ IngestJob: compare Parquet schema for target_date vs stored schema snapshot
  → PASS: proceed
  → FAIL: status="blocked_by_quality", halt, fire freshness alert
→ DataQualityService: row_count_stability, null_rate, schema_match, freshness, dedup
  → All PASS: proceed to compute
  → BLOCKING FAIL: log data_quality_checks, pipeline_run_log status="blocked_by_quality", halt
  → WARNING: log, continue with warning flag
→ MetricComputeService: DuckDB SQL per active metric → write metric_results
  → Success: pipeline_run_log status="success"
  → Error: log error_message, retry (max 3, exponential backoff)
→ AnomalyDetectionService: DOW-adjusted Z-score + EWMA per metric with ≥28 days history
  → Threshold exceeded: insert anomaly_alerts, compute attribution
→ ConflictDetectorService: compare active metric pairs, flag divergence > 5%
→ SystemHealthReportService: write daily summary row
```

Flow 2: Quality Failure Recovery

```
Quality check blocks Day 15 compute (null_rate = 0.38)
→ pipeline_run_log: status="blocked_by_quality"
→ data_quality_checks: passed=FALSE, severity="blocking"
→ Alert: metric freshness SLA breach
→ [Upstream logging bug found, corrected data arrives Day 16]
→ BackfillService: trigger_reason="quality_failure_recovery", affected_dates=["2024-01-15"]
→ Ingest + quality check for Day 15 backfill: passes
→ MetricComputeService recomputes Day 15: metric_results with late_arrival_flag=TRUE
→ pipeline_run_log: new backfill entry, triggered_by="backfill_job"
```

Flow 3: Metric Definition Version Change

```
active_users_v2 required
→ MetricRegistryService: create new metric_definition version=2, parent_version_id=v1.id
  → compute new config_hash
  → mark v1 is_active=FALSE, deprecated_at=now()
→ BackfillService: trigger_reason="definition_version_change", affected_dates=[90-day range]
  → Recompute all 90 days of v2 on historical Parquet
  → Write metric_results with metric_version=2
→ ConflictDetectorService (nightly): v1 vs v2 divergence detected
  → Inserts metric_conflicts record
→ API: /metrics/{id}/versions endpoint shows lineage; /conflicts shows active conflicts
```

Flow 4: Experiment Evaluation

```
Analyst submits EXP-001 evaluation at Day 14
→ ExperimentEvalService:
  1. Load treatment/control events from Parquet by experiment_id + variant
```

```

2. Compute per-user metric values (numerator, denominator, ratio)
3. Load X_pre (same metric, 14 days before experiment start)
4. apply_cuped() → y_adjusted, theta, variance_reduction_pct, edge_case, fallback
5. Welch t-test on CUPED-adjusted treatment vs control
6. Compute 95% CI
7. Check peeking: load prior evaluations, compare significance conclusions
8. Segment analysis across country, platform dimensions
9. Simpson's paradox check: overall direction vs segment directions
10. Novelty check: week-1 vs week-2 lift ratio
→ GuardrailGateService:
  For each guardrail_metric_config:
    evaluate_guardrail() → "clear" | "violated" | "immature"
→ DecisionEngineService:
  compute_recommendation() → guardrails first, then primary metric
→ Write experiment_evaluations record
→ Return evaluation detail + recommendation to API
→ Human reviewer: submit decision via /decisions endpoint
→ API enforces: if human_decision != system_recommendation → override_reason required
→ Write decision_log (append-only)

```

Flow 5: Post-Launch Validation

```

[At post_launch_validation_due date – triggered by APScheduler]
→ PostLaunchValidationService:
  1. Query decision_log for entries where post_launch_validation_due = today()
  2. For each:
    a. Compute actual post-launch primary metric change (ship_date to today)
    b. Compare to pre-ship prediction CI from experiment_evaluations
    c. For each delayed guardrail: re-run evaluate_guardrail() with mature data
  3. If delayed guardrail now violated:
    → Insert anomaly_alerts entry (detection_method="post_launch_validation")
    → Flag in system_health_reports: post_launch_validation_failures += 1
  4. Write calibration note to decision_log
  5. Update system_health_reports: calibration_score (positive prediction vs positive realization rate)

```

Flow 6: Anomaly Triage

```

Anomaly alert fires: DAU drop Day 28
→ anomaly_alerts: acknowledged=FALSE
→ Analyst reviews via UI: sees metric value, expected value, Z-score, attribution segment
→ Analyst action A (confirm): acknowledged=TRUE, false_positive=FALSE
→ Analyst action B (mark FP): acknowledged=TRUE, false_positive=TRUE, feedback_notes="Expected: iOS release rollout"
→ System: update rolling false_positive_rate in system_health_reports
→ If false_positive_rate > 0.40: flag in system_health report for threshold review

```

14. Security, Privacy, and Robustness

PII Assumptions

All data is synthetic. No real user identifiers, emails, names, or behavioral data anywhere in the system. Entity IDs are random UUID-format strings. The README explicitly states: "All event data is synthetically generated. No real user data was used in this project."

Synthetic Data Boundary

The `simulation_events` table documents every injected failure scenario and every synthetic decision. Anyone reviewing the codebase can trace exactly what was injected and when. The demo seed script is fully reproducible from a fresh database. There is no pretense that operational history was generated by real use.

Secrets and Configuration

All environment variables (`DATABASE_URL`, Sentry DSN, API keys if any) stored in `.env` (gitignored). `.env.example` committed. `docker-compose.yml` uses environment variable substitution. No secrets in repository.

API Authentication

FastAPI backend uses `x-API-Key` header for write operations (creating experiments, submitting decisions, acknowledging alerts). Read endpoints are unauthenticated for demo purposes. Documented in README as demo-only — not enterprise RBAC.

Auditability

- `pipeline_run_log`: append-only. No row is ever updated — retry creates a new row.
- `decision_log`: append-only. No decision can be deleted or modified.
- `metric_results`: immutable per `run_id`. Recomputed results create new rows with `late_arrival_flag = TRUE`.
- `config_hash`: changing a metric definition changes its hash, making any compute result traceable to the exact definition in effect.

Rollback and Reproducibility

Any pipeline run reproducible by: (1) identify `config_hash` from `pipeline_run_log`, (2) look up `metric_definitions` row, (3) re-run DuckDB query against same Parquet partition. The `simulation_events` table makes the demo state exactly reproducible from a fresh database using `seed_demo.py`.

Reviewer Identity

Decision log entries use fictional analyst names (`priya_sharma`, `james_okoye`, `sara_lindqvist`), defined in `DEMO_REVIEWERS` constant in the seed script and documented as fictional in README.

What Is Not Enterprise-Grade

- No SSO or RBAC
- No row-level security in Postgres
- No encrypted-at-rest Parquet
- Append-only enforced by application logic, not DB constraints
- No audit log immutability at storage level

These gaps are documented in the README. Interview answer: "The current implementation enforces append-only through application logic; in production, you'd use database-level constraints, a write-once storage layer for Parquet, and an access control system. I prioritized demonstrating the design intent over implementing enterprise-grade enforcement."

15. Cost, Latency, and Scale Story

Demo Scale (< 5M events/day)

DuckDB in-process handles all analytical compute. DAU metric over 90 days at 1M events/day: 200ms–1s. Parquet partitioned by date: 90-day queries scan 90 partition files. Storage: 1M events/day × 90 days × ~200 bytes compressed ≈ 3GB. All services in Docker containers on a single machine.

10x Scale (50M events/day)

DuckDB still works but compute time 5–30s for full recomputes. Mitigation: pre-aggregate metric results to daily level; UI reads from metric_results Parquet, not raw events. Add secondary partition by country (most queries filter on country). Redis (Upstash free tier) for pre-computed time series: UI read latency 5–20ms. Compute jobs move to separate worker process.

100x Scale (500M events/day)

DuckDB breaks on single-node compute. Transition: move raw Parquet to S3. Replace DuckDB with Trino or Spark. The dbt SQL models are portable — same SQL runs on DuckDB (dev), Trino (medium), Spark (large) without rewriting business logic. This is the architectural discipline: SQL-portable transforms, swappable compute engine. FastAPI continues to serve pre-computed aggregates — no change to API layer.

What Breaks First (In Order)

1. DuckDB in-process memory on large joins → move compute to separate worker / Spark
2. Single Parquet partition too large for efficient predicate pushdown → finer partitioning
3. Postgres pipeline_run_log grows large → partition by date, archive old records
4. Anomaly detection loop across many metrics in daily batch → parallelize via multiprocessing
5. APScheduler reliability at high frequency → transition to Prefect/Airflow

What Stays the Same

- SQL-defined metric logic in dbt (portable across compute engines)
- Postgres schema for registry, audit, decisions (scales to millions of rows)
- FastAPI read layer (reads pre-computed aggregates, not raw events)
- CUPED, Welch t-test, guardrail gate logic (stateless, fast)

16. Demo Narrative

Command Sequence

```
BASH
# 1. Start infrastructure
docker-compose up -d

# 2. Run migrations
alembic upgrade head

# 3. Generate 60-day operational history
python seed_demo.py --scenario full_demo --start-date 2024-01-01

# 4a. View via thin UI
open http://localhost:3000

# 4b. View via terminal report
python show_demo_report.py
```

The 60-Day Operational Story

Days 1–10: Baseline

Clean data. All pipelines succeed. Metrics compute daily. No anomalies. DOW baseline established. `system_health_reports: pipeline_success_rate = 1.0, anomaly_alerts_fired = 0.`

Day 11: Schema Drift Blocks Pipeline

`pipeline_run_log: status = "blocked_by_quality". data_quality_checks: check_name = "schema_match", passed = FALSE, check_details = {"missing_columns": ["country"]}`. Metric freshness alert fires.

Day 12: Recovery and Backfill

Schema corrected. Two `pipeline_run_log` entries for Day 11: original failed + backfill success. `metric_results` for Day 11: `late_arrival_flag = TRUE`. Normal Day 12 compute.

Day 15: Null Spike

`data_quality_checks: null_pct = 0.38`, blocking. Compute halted. Recovered Day 16 with backfill.

Day 20: Late-Arriving Data Recompute

Two `metric_results` rows for Day 19: original (1.2M rows) and recomputed (1.38M rows, `late_arrival_flag = TRUE`).

Day 25: Double-Counting

Row count 3.1× 7-day average. Blocking dedup check. Resolved Day 26.

Day 28: Real Anomaly — iOS Outage

DOW-adjusted Z-score = 3.8. `anomaly_alerts: severity = "high", attribution = {"top_segment": "platform=ios", "contribution": 0.81}`. Analyst `priya_sharma` confirms. `false_positive = FALSE`.

Day 30: EXP-001 Starts (Checkout Redesign)

Experiment registered. MDE calculator confirms experiment is adequately powered: "Expected effect of +3.5% exceeds minimum detectable effect of +1.4% at 80% power over 14 days."

Day 35: DOW False Positive — Correctly Suppressed

Long weekend. Raw Z-score = 2.7. DOW-adjusted Z-score = 1.1. Alert does NOT fire. *"This is why DOW adjustment exists."*

Day 40: Metric Definition Change

`active_users_v2` created. Backfill 90 days. ConflictDetector fires: `value_divergence_pct = 0.23`. Three conversion rate conflicts detected nightly. UI shows all three pairs in `/conflicts`.

Day 44: EXP-001 Evaluation — Wait for Guardrail Maturity

CUPED: `theta = 0.61, variance_reduction_pct = 31.2%`. Purchase rate: `p = 0.023, CI = [+1.2%, +5.8%]`. `page_load_time` guardrail: mature, cleared. `return_rate` guardrail: `status = "immature"` (day 14 of 60-day window). Decision: `"wait_guardrail_maturity"`.

Day 44: EXP-002 Human Override

System: "hold" (`p=0.11`). `james_okoye: human_decision = "override_ship", override_reason = "Week 4 campaign confounded overall lift; segmented analysis shows +5.1% for returning users (p=0.009). PM accepted risk."` Logged, immutable.

Day 50: EXP-003 Clean Ship

`p=0.008, CI=[+2.1%,+7.3%]`, guardrails mature+cleared, CUPED applied, no flags. `sara_lindqvist: human_decision = "ship". post_launch_validation_due` set to Day 80.

Day 60: EXP-001 Retroactive Guardrail Breach

PostLaunchValidationService runs. return_rate mature CI = [+0.4%, +2.1%], tolerance = 1.5%. Retroactive violation. anomaly_alerts entry: detection_method = "post_launch_validation". system_health_reports: post_launch_validation_failures = 1.

"The system recorded the right uncertainty at Day 44. The human reviewer accepted it. Now the consequence is visible and logged."

2-Minute Narration Script

"MetaSignal simulates 60 days of platform operation. I'll narrate the key moments.

Days 1–10 are clean baseline — the system needs this to calibrate its anomaly detector. On Day 11, a schema drift blocks the pipeline — the country column goes missing from an upstream Parquet file. The pipeline doesn't silently compute wrong data; it halts, logs the failure, and fires a freshness alert. The backfill on Day 12 recomputes Day 11's metrics with late_arrival_flag=TRUE.

Day 28 shows a real anomaly — DAU drops 14% due to an iOS outage. The DOW-adjusted Z-score is 3.8. More interesting is Day 35: a long weekend creates a 7% DAU drop that the raw Z-score would fire on. The DOW-adjusted score is 1.1 — below threshold. No alert. That suppression is the point.

Day 40 shows metric conflict detection. We create active_users_v2 with a different filter, and the conflict detector flags a 23% divergence between v1 and v2 nightly. Three conversion rate definitions exist in the registry with different denominators — all three conflict pairs fire.

The most important scenario is EXP-001. At Day 44, the primary metric is significant — CUPED reduced variance by 31%, $p=0.023$. But the return rate guardrail is immature; it takes 60 days to mature. The system recommends 'wait for guardrail maturity' rather than ship. At Day 60, the mature return rate shows a 0.4–2.1% increase against a 1.5% tolerance. Retroactive guardrail violation, logged.

Every decision is auditable. EXP-002 shows a human override with a written business justification. EXP-003 is a clean ship. All three outcomes — wait, override, ship — are in the decision log with full context."

17. Must-Have / Should-Have / Optional Scope**Must-Have (Project Not Serious Without These)**

- Postgres schema with all tables from Section 7, Alembic migrations
- Failure-injected synthetic data generator (all 13 scenarios from Section 8)
- DataQualityService with blocking failures that actually halt compute
- pipeline_run_log with real status = "blocked_by_quality" entries
- MetricComputeService with DuckDB + dbt on Parquet
- BackfillService with late_arrival_flag = TRUE on recomputed rows
- 6 metric definitions with explicit denominator_sql, config_hash, versioning
- 3 conflicting metric pairs in the registry (ConflictDetector must fire on demo data)
- CUPED with all four edge case handlers in actual code
- A/A validation script (1000 runs, result committed to README)
- Welch t-test with CI computation
- GuardrailGateService with gate-before-primary ordering (Scenario 2 in golden suite must pass)
- Right-censoring maturity window logic (Scenario 8 must pass)

- PostLaunchValidationService — Day 60 retroactive breach scenario (Scenario 9 must pass)
- decision_log with override_reason API enforcement (Scenario 12 must pass)
- Peeking warning detection
- DOW-adjusted Z-score anomaly detection
- Anomaly attribution (top-k segment contribution)
- Golden scenario suite in CI (all 12 cases)
- system_health_reports daily summary
- simulation_events table (reproducibility proof)
- Demo runner script (seed_demo.py producing consistent 60-day history)
- show_demo_report.py (terminal evidence summary for narration)
- Docker + docker-compose
- README with: A/A false positive rate result, anomaly precision/recall (labeled as synthetic), pipeline run history sample table

Should-Have (High Hiring Signal)

- EWMA anomaly detection (complement to Z-score; medium alert threshold requires both)
- Novelty effect detection
- Peeking warning evaluation history chart
- FastAPI with Pydantic schemas (all endpoints)
- Thin frontend: metric timeseries, anomaly feed, experiment registry, decision log
- dbt Core for versioned SQL transforms and test assertions
- Structured JSON logging (all services)
- GitHub Actions CI (golden suite + A/A validation + dbt tests)
- Segment analysis in experiment evaluation (Simpson's paradox detection)
- Sentry free tier (exception tracking only)

Optional / Stretch

- Redis caching (Upstash free tier) — meaningful at 50M+ events, not at demo scale
- Prefect orchestration upgrade from APScheduler
- Prophet-based anomaly detection upgrade from EWMA
- Sequential testing / mSPRT upgrade from one-shot t-test
- Vercel/Render deployment of frontend
- LLM-assisted metric conflict explanation (one node only — natural language SQL diff explanation)

Cut First

- LLM conflict explanation node — zero hiring signal from the LLM part; signal is in the detection
- Redis caching — in-memory dict sufficient for demo
- Sequential testing — CUPED + one-shot t-test is already differentiated

Never Cut

- The three conflicting metric pair scenario with ConflictDetector actually firing (if it doesn't fire on demo data, the denominator story is just a claim)
- A/A validation with measured result in README (if you claim false positive rate control, you must show it)
- The delayed guardrail retroactive breach at Day 60 (most unique and interview-differentiating scenario in the project)
- Decision log override with written override_reason (most human and memorable moment in the demo)
- pipeline_run_log with real failure entries (if every row is "success", operational history is hollow)
- Guardrail-before-primary ordering in decision engine (the technical correction from v1 that makes the logic correct)

18. Honest Interview and Resume Positioning

Resume Bullets

- Built MetaSignal, a solo production-simulated metrics intelligence and experiment decision platform over partitioned Parquet event data (DuckDB + dbt); designed denominator-explicit metric registry with full version lineage, config-hash-tracked compute audit trail, and a conflict detector that nightly flags denominator divergence — three definitions of "conversion rate" with different denominators all active and conflicting
- Implemented CUPED variance reduction with four edge case handlers (weak covariate, missing pre-experiment data, zero variance in X_{pre} , imbalanced arms); validated false positive control via 1000 synthetic A/A runs — p-value distribution KS test against Uniform[0,1] passes; variance reduction measured per experiment and stored in evaluation record
- Designed guardrail-first experiment decision engine with right-censoring for delayed metrics (60-day return rate maturity window); simulated retroactive guardrail breach where 14-day readout appeared safe but 60-day matured result violated 1.5pp tolerance; all decisions logged with mandatory override justification in append-only audit trail
- Engineered 60-day production-simulated pipeline with failure injection (schema drift, null spikes, late-arriving data, double-counting); pipeline_run_log shows blocked runs, exponential backoff retries, and backfill recovery; DOW-adjusted anomaly detector validated on labeled synthetic set (20 events, deliberate difficulty distribution) — results reported as synthetic-validated, not production-claimed

30-Second Pitch

"MetaSignal is a metrics intelligence and experiment decision platform that I built to demonstrate production-grade thinking about analytics infrastructure. It's a solo non-production project, but it simulates 60 days of real operational history. The system enforces denominator-explicit metric definitions — three definitions of 'conversion rate' with different denominators all exist in the registry, and the conflict detector flags all three pairs nightly. Experiments are evaluated with CUPED and pre-defined guardrail gates that are evaluated before the primary metric, not after. The most interesting scenario is a checkout experiment that looks clean at 14 days but has a retroactive guardrail breach at 60 days when return rate matures. Every design decision is defensible at the mechanism level."

2-Minute Architecture Explanation

See Section 16 narration script.

What Can Honestly Be Claimed

- The statistical implementation is correct: CUPED formula validated on 1000 synthetic A/A runs with measured false positive rate
- Guardrail logic is correctly ordered: gates evaluated before primary metric; delayed guardrails handled with maturity window
- The operational history is scripted and labeled: 60 days of simulated events with real failure modes encountered and handled in the logs
- Anomaly precision/recall measured on labeled synthetic set with honest difficulty distribution — labeled as such
- The denominator design story is demonstrated by conflicting metric pairs that actually cause the conflict detector to fire

What Must Not Be Claimed

- "This is a production system" — it is not
- "Precision/recall numbers are from production data" — they are from synthetic labeled events
- "Human override decisions were made by real analysts" — they were written by the builder
- "Production-ready" — the correct term is "production-simulated"

How to Explain Production-Simulated Without Sounding Fake

"The distinction I draw is between production-shaped — the architecture looks right — and production-simulated — the system has actually encountered and handled realistic failure modes, and the evidence is in the logs. In MetaSignal, I scripted a 60-day operational scenario where the system encounters schema drift, quality failures, late-arriving data, a guardrail breach that only becomes visible after 60 days, and a human override decision. All of these appear as real database entries you can query. The data is synthetic, the decisions were written by me, and the users don't exist — but the system's response to these scenarios is real, correct, and auditable. I always lead with that framing when discussing the project."

Signal Level

MetaSignal signals **senior-to-staff-leaning DS** capability:

- **Senior signal:** Correct statistical implementation (CUPED, correct multiple comparison discipline, guardrail gate logic), data quality pipeline with blocking failures, metric versioning and lineage
- **Staff signal:** Denominator design as a first-class product concern, decision audit trail as a measurable system health metric, right-censoring awareness for delayed guardrails, operational history design, 60-day retrospective validation loop

Combined with DevPulse, the total signal is **staff-level** across both AI engineering and analytics/decision systems — a combination that specifically targets FAANG L5/L6 DS and Product Scientist roles.

19. Red-Team Review

What Could Still Look Fake

The operational history is too clean despite failures. If every retry succeeds on the first attempt and every backfill completes in one run, it looks scripted. Mitigation: add at least one scenario where `retry_count = 2` before success, one backfill job that itself fails and is re-triggered, one schema drift that takes 2 days to resolve.

The anomaly attribution is too tidy. "`contribution`": **0.81** for a single segment looks suspiciously clean. The attribution JSONB should show multi-segment cases and noisy cases alongside clean attributions.

The override reasons are too articulate. Mix in one brief override reason ("PM approved via Slack, escalation in progress") alongside the well-reasoned ones.

The ConflictDetector may not fire. If the divergence threshold is miscalibrated, the three designed conflict pairs may not reach 5% divergence on the synthetic data. Test this explicitly before finalizing the demo — run the detector and verify `metric_conflicts` rows are inserted.

The A/A validation may not be run before the interview. If you claim false positive rate control but haven't run the 1000-trial A/A test, you cannot quote the number. Run it. Commit the output to the README.

What Could Break in an Interview

"Walk me through CUPED edge case handling in your code." The edge case logic must be in the actual code, not just in this PRD. If it's described but not implemented, an interviewer who asks to see the code will find the gap.

"What is your actual false positive rate?" The answer must be specific: "On 1000 A/A runs with synthetic data, X% had $p < 0.05$ at $\alpha=0.05$. The KS test p-value against Uniform(0,1) was Y." If you can't answer precisely, you cannot claim false positive rate control.

"Why is your guardrail tolerance 1.5pp? How would you set it in a real system?" Answer: "In the simulation, 1.5pp is chosen against a baseline return rate of ~5%. In a real system, this tolerance is a product decision based on the cost model — how much incremental return rate is acceptable in exchange for how much primary metric improvement. The `guardrail_tolerance_pct` field is explicitly a configuration, not a statistical parameter, because the threshold represents the business's risk appetite, not a statistical property of the data."

"Your guardrails don't use Bonferroni — why not?" Answer: "Guardrails are mechanical gates, not hypothesis tests. Bonferroni corrects for multiple comparisons in hypothesis testing — it raises the significance threshold to control family-wise error rate. Applying Bonferroni to guardrails would lower the sensitivity of the safety mechanism by requiring stronger evidence of harm before blocking a ship decision. For a safety gate, that's the wrong direction — harm is asymmetric. We evaluate guardrails against a pre-defined tolerance threshold, which is a product decision, not a statistical inference."

"What if the pre-experiment covariate for CUPED is negatively correlated with the outcome?" Answer: "Negative theta is fine — CUPED works regardless of sign if the correlation is strong. A negative theta means the pre-experiment covariate moves opposite to the post-experiment outcome. The CUPED adjustment subtracts $\theta \times (X_{pre} - \text{mean}(X_{pre}))$ from Y_{post} , and with negative theta, users with above-average X_{pre} get upward adjustment. The edge case check is on the magnitude of theta (< 0.05), not its sign."

What Must Be Tested Before Showing Anyone

1. A/A test: run 1000 trials, measure false positive rate, verify KS test, commit histogram to README
2. ConflictDetector: run on demo data, verify `metric_conflicts` rows inserted for all three designed conflict pairs
3. Schema drift scenario: run `seed_demo.py`, query `pipeline_run_log`, verify `status = "blocked_by_quality"` on Day 11
4. Delayed guardrail breach: verify Day 60 `PostLaunchValidationService` fires and inserts `anomaly_alerts` entry for EXP-001
5. Golden scenario suite: all 12 cases pass CI
6. Override reason enforcement: verify API returns 422 when `override_reason` is NULL with an override decision
7. DOW suppression: verify Day 35 anomaly does NOT fire (DOW-adjusted Z-score below threshold)

How to Defend the Project Honestly

"I'll be direct: this is a solo non-production project with synthetic data. What I can defend: the statistical implementation is correct and validated — CUPED false positive rate confirmed via A/A testing, anomaly detection labeled and honest about being synthetic-validated. The system has logged operational history that includes real failure modes and real recoveries — you can query the database. The design decisions are defensible at the mechanism level. What I can't claim is production scale, real users, or enterprise-grade security. I chose to simulate operational depth and statistical rigor over building a polished frontend or claiming production credibility I don't have."

20. Recommendation Before Execution Plan

Timeline: 4 Weeks

Why not 3 weeks:

A clean 14-day implementation of the core capabilities (metric compute, anomaly detection, experiment evaluation, decision log) is achievable in 3 weeks. But "production-simulated" adds concrete work not in a clean build:

Additional work	Estimate
Failure injection design + generator (13 scenarios)	2–3 days
CUPED edge case handling + A/A validation script	1–2 days
Right-censoring + PostLaunchValidationService	1–2 days
Golden scenario suite (12 cases) in CI	1 day
seed_demo.py producing coherent 60-day history	2 days
ConflictDetector threshold calibration on demo data	1 day
Statistical debugging (CUPED FP drift, anomaly thresholds)	2–3 days
README evidence tables + A/A result commit	1 day
Total additional	11–15 days

4 weeks (28 days) with the first 2–3 days on data generator design and failure scenario specification (before coding begins) is the right estimate.

What Must Not Slip

1. A/A validation script with committed result — without this, the CUPED false positive claim cannot be made
2. pipeline_run_log with real `blocked_by_quality` and `failed` entries — if every row is "success", operational history is hollow
3. Delayed guardrail retroactive breach (Day 60 scenario) — the most unique, interview-differentiating capability in the project
4. Three conflicting metric pairs with ConflictDetector actually firing — if this is abstract, the denominator story is just a claim
5. Golden scenario suite passing in CI — without this, the evaluation engine has no regression protection
6. Override reason enforcement in the API — if not enforced in code, the human-in-the-loop story is a UI field with no teeth
7. Guardrails evaluated before primary metric in the decision logic (Scenario 2 must pass)

What Can Be Mocked

- Redis caching (use in-memory dict)
- Sentry integration (use structured logging only)
- Sequential testing / mSPRT
- LLM conflict explanation node (cut entirely)
- Prefect (APScheduler sufficient)
- Frontend polish (minimal Recharts is sufficient)
- dbt production run mode (use dbt for SQL versioning; DuckDB is primary compute path)

What Must Be Implemented For Real

- CUPED with edge case handlers in actual code (not just documented)

- A/A validation with measured result committed to README
 - GuardrailGateService with correct ordering (guardrails before primary)
 - RightCensoringService with maturity window status logic
 - PostLaunchValidationService triggered at maturity date
 - DataQualityService with blocking failures that halt compute (pipeline_run_log must show blocked entries)
 - BackfillService creating `late_arrival_flag = TRUE` rows (not just a claim)
 - ConflictDetectorService inserting metric_conflicts rows on demo data (calibrated and tested before demo)
 - Decision log with override_reason enforcement at the API layer (422 on missing reason)
 - DOW-adjusted Z-score (not raw Z-score — Day 35 false positive must be suppressed)
 - simulation_events seed table (reproducibility proof)
 - show_demo_report.py with evidence summary output (pipeline failures, anomaly alerts, decision log, validation failures)
-

MetaSignal PRD v2.0 — April 2026

Solo-built, non-production, production-simulated

Targets: Senior/Staff DS · Product Scientist · Decision Systems — FAANG-level

Companion project: DevPulse (AI engineering, retrieval systems, LLM orchestration)